

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

NGUYỄN THẠCH HÀ ANH
MSHV: 25C15001

GIẢI PHẪU MULTILAYER PERCEPTION: TÍCH HỢP LOGIC NỘI SUY PARABOL TRÊN NỀN TẢNG PYTORCH

Báo cáo Lab 01
Toán cho trí tuệ nhân tạo nâng cao – HP1-K35-TTNT

TP. HỒ CHÍ MINH — 2026

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

GIẢI PHẪU MULTILAYER PERCEPTION: TÍCH HỢP LOGIC NỘI SUY PARABOL TRÊN NỀN TẢNG PYTORCH

Báo cáo Lab 01

Toán cho trí tuệ nhân tạo nâng cao – HP1-K35-TTNT

NGUYỄN THẠCH HÀ ANH

MSHV: 25C15001

Đơn vị công tác, chức vụ

hoặc ngành đang được đào tạo

Học viên Cao học

chuyên ngành Trí tuệ nhân tạo

TP. HỒ CHÍ MINH — 2026

MỤC LỤC

PHẦN MỞ ĐẦU	1
Chương 1 NỀN TẢNG TỐI ƯU SỐ VÀ THUẬT TOÁN HEURISTIC TRONG KHÔNG GIAN LIÊN TỤC	3
1.1. Hàm Lỗi và Các Tiêu chí Nhận diện	3
1.2. Tối ưu Không lỗi và Thuật giải Mẹo (Heuristic)	3
Chương 2 KHẢO SÁT CẤU TRÚC VÀ CƠ CHẾ TOÁN HỌC CỦA MẠNG NƠ-RON ĐA LỚP (MLP)	5
2.1. Kiến trúc Phân tầng	5
2.2. Quá trình Lan truyền Tiến (Forward Propagation)	6
Chương 3 GIẢI PHẪU THUẬT TOÁN LAN TRUYỀN NGƯỢC (BACKPROPAGATION)	8
3.1. Đạo hàm tại Lớp Đầu ra (Cập nhật ma trận B)	8
3.2. Đạo hàm tại Lớp Ẩn (Cập nhật ma trận A)	8
3.3. Cơ chế Hoạt động trong Trình Tối ưu hóa (Optimizer) của PyTorch	9
Chương 4 TỐI ƯU HÓA NÂNG CAO: CHUYỂN DỊCH TỪ GRADIENT DESCENT SANG PHƯƠNG PHÁP PARABOL	11
4.1. Cơ sở Toán học của Phương pháp Parabol (Brent's Method).....	11
4.2. Ứng dụng Parabol vào Mạng Nơ-ron: Thuật toán QuickProp	11
4.3. So sánh Tương quan Đặc tính Toán học.....	12
Chương 5 TÍCH HỢP VÀ THAY THẾ LOGIC PARABOL VÀO MÃ NGUỒN THỰC TẾ	14
5.1. Khởi tạo Bức tranh Quá khứ	14
5.2. Lan truyền Tiến và Kích hoạt Autograd	14
5.3. Triển khai Logic Toán học Cập nhật Parabol.....	14
KẾT LUẬN.....	16
DANH MỤC TÀI LIỆU THAM KHẢO	17

PHẦN MỞ ĐẦU

Giải quyết vấn đề (problem solving) bằng máy tính, xét về bản chất, là quá trình lập trình (programming) để máy tính thực thi tự động các thao tác nhằm tìm ra lời giải cho vấn đề được cho. Tiến trình này đòi hỏi sự đặc tả (specification) rõ ràng thông qua dữ liệu đầu vào (input), dữ liệu đầu ra mong muốn (output) và các ràng buộc đi kèm. Dưới góc độ ngôn ngữ của Khoa học máy tính (computer science), lập trình bao hàm việc tổ chức, quản lý dữ liệu thông qua cấu trúc dữ liệu (data structure) và sắp xếp các chỉ thị theo một trình tự tất định thông qua thiết kế thuật giải (algorithm). Sự hội tụ của hai yếu tố này đã được nhà khoa học máy tính Nikolaus Wirth công thức hóa thành một nguyên lý kinh điển: Chương trình = Cấu trúc dữ liệu + Thuật giải.

Bằng cách biểu diễn vấn đề cần giải quyết P dưới dạng một mệnh đề toán học $P: D \rightarrow \{False, True\}$, việc giải quyết vấn đề thực chất là việc tìm kiếm một lời giải s thuộc không gian miền D sao cho mệnh đề $P(s)$ đạt giá trị $True$. Dựa trên nền tảng này, bài toán giải quyết vấn đề được chuyển hóa thành bài toán tìm kiếm (search): $search(s \in D): P(s)$. Tùy thuộc vào bản chất của miền D , các phương pháp tiếp cận sẽ có sự khác biệt rõ rệt. Khi D là một miền hữu hạn, các công cụ từ toán học tổ hợp (combinatorics) và lý thuyết đồ thị (graph theory) có thể được áp dụng để thực hiện quá trình tìm kiếm chứng minh (search_2), qua đó thiết lập một lộ trình (path) từ trạng thái ban đầu s_0 đến trạng thái đích s^* thông qua các quan hệ bắc cầu hai ngôi.

Tuy nhiên, trong bối cảnh của Trí tuệ nhân tạo (AI) và Học máy (Machine Learning), không gian D thường là một miền liên tục (continuous domain) với vô số các điểm dữ liệu. Việc tìm kiếm một lời giải chính xác tuyệt đối trong một tập hợp có kích thước vô cùng là điều bất khả thi về mặt lý thuyết cũng như thực tiễn, bởi xác suất tìm trúng có thể được hiểu là $1/\infty$. Do đó, thay vì tìm kiếm lời giải chính xác tuyệt đối, các hệ thống AI chuyển sang giải quyết bài toán bằng cách xấp xỉ (approximation) thông qua các kỹ thuật mô phỏng. Tiêu chí tìm kiếm lúc này được nói lỏng thành việc tìm kiếm giá trị x sao cho $f(x) \approx 0$ hoặc chịu một mức sai số tối đa ϵ ($|f(x)| \leq \epsilon$).

Đối với các bài toán xử lý dữ liệu lớn (big data), khi mà kích thước miền xác định là vô cùng đồ sộ, nhiệm vụ cốt lõi là phát hiện ra "quy luật" ẩn tàng bên trong tập dữ liệu quan trắc (observations): $\Omega = \{(X^{(i)}, y^{(i)}) \in D \times R\}_{i=1}^N$. Quy luật này được biểu diễn dưới dạng một hàm thực nhiều biến $f: D \rightarrow R$, ánh xạ véc-tơ đầu vào (biến độc lập) X thành một giá trị đầu ra (biến phụ thuộc) y . Thay vì rà soát trong toàn bộ không gian các hàm thực liên tục, giới hạn tìm kiếm được thu hẹp vào không gian các hàm có dạng xác định trước, được chi phối bởi tập tham số (trọng số) $w \in \mathbb{R}^m$. Bài toán mô hình hóa dữ liệu chính thức

được chuyển đổi thành bài toán tối ưu hóa (optimization problem) với mục tiêu tìm kiếm tập tham số w^* sao cho hàm mất mát (loss function) $w^* = \min_w \mathcal{L}(w)$ đạt giá trị cực tiểu:

$$w^* = \min_w L(w)$$

$$\mathcal{L}_\Omega(w) = \frac{1}{N} \sum_{i=1}^N |f_w(x^{(i)}) - y^{(i)}|$$

Việc giải quyết bài toán tối ưu hóa này là nền tảng chi phối mọi thuật toán học máy, từ các mô hình hồi quy tuyến tính (linear regression) truyền thống cho đến các mạng nơ-ron học sâu (Deep Learning) đa tầng phức tạp.

Chương 1

NỀN TẢNG TỐI ƯU SỐ VÀ THUẬT TOÁN HEURISTIC TRONG KHÔNG GIAN LIÊN TỤC

Trong trường hợp hàm mục tiêu $\mathcal{L}: \mathbb{R}^m \rightarrow \mathbb{R}$ cần được cực tiểu hóa trên miền xác định D , tiến trình tìm kiếm lời giải phụ thuộc mật thiết vào hình thái hình học của hàm số. Nếu hàm $\mathcal{L}$ không phải là hàm đơn điệu, cực trị của nó sẽ xuất hiện tại những điểm mà đạo hàm bậc nhất theo mọi biến thành phần bằng không, tức là nghiệm của hệ phương trình $\frac{\partial \mathcal{L}}{\partial x_i}(x) = 0$ với mọi $i = 1, 2, \dots, n$.

1.1. Hàm Lồi và Các Tiêu chí Nhận diện

Tối ưu hóa có thể được phân làm hai lớp chính: tối ưu lồi (convex optimization) và tối ưu không lồi (non-convex optimization). Một miền D được định nghĩa là miền lồi nếu với mọi điểm $x, y \in D$ và với mọi $\alpha \in [0, 1]$, điểm tổ hợp tuyến tính $\alpha x + (1 - \alpha)y$ cũng hoàn toàn nằm trong D . Dựa trên định nghĩa này, một hàm liên tục $f(x)$ xác định trên miền lồi D được gọi là hàm lồi nếu nó thỏa mãn bất đẳng thức:

$$f(\alpha x + (1 - \alpha)y) \leq \alpha f(x) + (1 - \alpha)f(y)$$

Ý nghĩa hình học của tính chất lồi cho thấy mọi điểm trên đồ thị của hàm số luôn nằm dưới đường thẳng nối hai điểm bất kỳ trên đồ thị đó. Sự xuất hiện của hàm lồi mang một giá trị cực kỳ to lớn trong toán học tối ưu, bởi mọi điểm cực tiểu cục bộ (local minimum) của hàm lồi cũng đồng thời là điểm cực tiểu toàn cục (global minimum). Bài toán tối ưu lồi luôn đảm bảo tồn tại lời giải ổn định và nhất quán.

Để đánh giá một hàm số có phải là hàm lồi hay không, toán học cung cấp các tiêu chuẩn dựa trên đạo hàm. Nếu hàm số có đạo hàm bậc hai liên tục (thuộc lớp C^2), ma trận Hesse (Hessian matrix) $\nabla^2 f(x)$ - chứa toàn bộ các đạo hàm riêng bậc hai - sẽ được sử dụng. Một hàm số là lồi khi và chỉ khi ma trận Hesse của nó là một ma trận xác định nửa dương (positive semidefinite matrix), tức là với mọi véc-tơ $h \in \mathbb{R}^m$, tích $h^T \cdot \nabla^2 f(x) \cdot h \geq 0$. Dựa trên định lý tối ưu lồi quan trọng, việc tìm kiếm nghiệm tối ưu x^* của hàm lồi tương đương với việc giải phương trình véc-tơ đạo hàm $\nabla f(x^*) = 0$.

Trong bối cảnh của học máy cơ bản, cụ thể là hồi quy tuyến tính ứng dụng kỹ thuật bình phương tối thiểu (Ordinary Least Squares - OLS), mô hình được giả định là một phép biến đổi tuyến tính $f_w(x) = w^T \cdot x$. Khi đó, hàm mất mát $\mathcal{L}(w) = \|A \cdot w - B\|^2$ được chứng minh là một hàm lồi (có dạng toàn phương) với ma trận Hesse $\nabla^2 \mathcal{L}(w) = 2(A^T \cdot A) \geq 0$. Do đó, nghiệm tối ưu có thể được xác định bằng công thức giải tích trực tiếp: $w^* = (A^T \cdot A)^{-1} \cdot A^T \cdot B$.

1.2. Tối ưu Không lồi và Thuật giải Mẹo (Heuristic)

Tuy nhiên, trong các bài toán thực tế phức tạp, khi dữ liệu quan trắc không có mối liên hệ tuyến tính, việc sử dụng các phép biến đổi tuyến tính thuần túy sẽ dẫn đến sai số

mô hình cực lớn. Mặc dù có thể sử dụng phương pháp thử dữ liệu để tìm tập các hàm cơ sở phi tuyến $f_i(x)$ nhằm tuyến tính hóa dữ liệu, việc lựa chọn bao nhiêu hàm và chọn như thế nào phụ thuộc rất lớn vào kinh nghiệm và tri thức chuyên gia của người phân tích.

Khi các mô hình học sâu ra đời nhằm tự động hóa việc trích xuất các đặc trưng phi tuyến này, cấu trúc hàm mục tiêu không còn là hàm lồi. Đối với một không gian hàm không lồi (non-convex), việc tìm nghiệm giải tích trực tiếp là bất khả thi. Thay vào đó, máy tính phải dựa vào các mẹo giải (heuristic) - những phương pháp tìm kiếm dựa vào kinh nghiệm lặp để trả về lời giải "tốt" chấp nhận được thay vì một lời giải chính xác tuyệt đối.

Một trong những chiến lược heuristic phổ biến là phương pháp tham lam (greedy) hoặc phương pháp tìm kiếm giảm đạo hàm (Gradient Descent - GD). Thay vì tìm kiếm ngẫu nhiên trong toàn bộ miền D , GD sử dụng thông tin từ véc-tơ đạo hàm $\nabla f(x)$ để hướng dẫn quá trình lặp. Véc-tơ đạo hàm luôn chỉ theo hướng tăng dốc nhất của hàm số; do đó, việc di chuyển ngược hướng với đạo hàm sẽ dẫn hệ thống trượt dần xuống điểm cực tiểu cục bộ (leo-đồi đảo ngược). Công thức lặp tổng quát của Gradient Descent được biểu diễn là:

$$w^{(t+1)} = w^{(t)} - \alpha \nabla \mathcal{L}(w^{(t)})$$

Trong đó, $\alpha \in (0,1]$ là hệ số học (learning rate), kiểm soát độ lớn của mỗi bước nhảy. Mặc dù Gradient Descent đảm bảo hệ thống sẽ hội tụ về một cực trị, nhưng nó bị giới hạn bởi tính tĩnh của hệ số α và không thể tận dụng được các "thông tin quá khứ" để gia tốc quá trình hội tụ. Sự thiếu linh hoạt này đã mở đường cho những thuật toán tối ưu tiên tiến hơn, điển hình là Phương pháp Nội suy Parabol.

Chương 2

KHẢO SÁT CẤU TRÚC VÀ CƠ CHẾ TOÁN HỌC CỦA MẠNG NƠ-RON ĐA LỚP (MLP)

Mạng nơ-ron truyền thẳng đa lớp (Feed Forward Multi-Layered Neural Network), hay Mạng Nơ-ron Đa Lớp (MLP), là kiến trúc tiêu biểu đại diện cho khả năng xấp xỉ hàm phi tuyến phức tạp trong Học sâu (Deep Learning). MLP ra đời để khắc phục giới hạn chỉ mạng của mô hình Perceptron một lớp - vốn chỉ giải quyết được các bài toán phân tách tuyến tính thuần túy và hoàn toàn thất bại trước các bài toán logic không tuyến tính đơn giản như cổng XOR.

2.1. Kiến trúc Phân tầng

Để tiến hành phân tích mã nguồn và cơ chế cập nhật tham số, báo cáo này lựa chọn mô hình MLP được xây dựng trên thư viện mã nguồn mở PyTorch để giải quyết bài toán phân loại tập dữ liệu chữ số viết tay MNIST. Cấu trúc của mạng bao gồm các tầng nơ-ron tuần tự:

1. **Lớp đầu vào (Input Layer):** Tiếp nhận véc-tơ đặc trưng 784 chiều, tương ứng với 784 điểm ảnh (pixel) từ một hình ảnh kích thước 28x28. Dữ liệu đầu vào $x \in \mathbb{R}^{784}$ được chuẩn hóa về phạm vi.
2. **Lớp ẩn (Hidden Layers):** Mạng có thể chứa một hoặc nhiều lớp ẩn. Trong phân tích toán học dưới đây, để đảm bảo tính tường minh, ta sẽ khảo sát một cấu trúc mạng 3 lớp (1 lớp ẩn) truyền thẳng. Giả sử lớp ẩn có J nơ-ron. Các trọng số kết nối từ lớp đầu vào đến lớp ẩn được biểu diễn bằng ma trận $A = [a_{ij}] \in \mathbb{R}^{(m+1) \times J}$, trong đó a_{0j} là tham số độ lệch (bias). Tại thực tế mã nguồn, số lượng nơ-ron có thể là 512 hoặc 256.
3. **Lớp đầu ra (Output Layer):** Bao gồm 10 nơ-ron tương ứng với 10 lớp phân loại (từ chữ số 0 đến 9). Trọng số từ lớp ẩn đến lớp đầu ra được biểu diễn bằng ma trận $B = [b_{jk}] \in \mathbb{R}^{(J+1) \times 10}$, với b_{0k} là độ lệch.

Sự kết hợp của kiến trúc này trong PyTorch được định nghĩa bằng cấu trúc lớp kế thừa `nn.Module`. Dưới đây là lược đồ mã nguồn thể hiện việc khởi tạo các ma trận trọng số (thông qua `nn.Linear`) và hàm phi tuyến (thông qua `nn.ReLU`):

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class SimpleMLP(nn.Module):
    def __init__(self):
        super(SimpleMLP, self).__init__()
```



```

# Lớp ẩn: Khởi tạo ma trận A (784 x 512) và vector bias a_0
self.hidden = nn.Linear(28 * 28, 512)
# Lớp đầu ra: Khởi tạo ma trận B (512 x 10) và vector bias b_0
self.output = nn.Linear(512, 10)

def forward(self, x):
    # Biến đổi vector ảnh 2D thành 1D
    x = x.view(-1, 28 * 28)
    # Lan truyền tiến qua lớp ẩn và hàm kích hoạt
    x = F.relu(self.hidden(x))
    # Lan truyền tiến qua lớp đầu ra
    x = self.output(x)
    return x

```

2.2. Quá trình Lan truyền Tiến (Forward Propagation)

Chức năng **forward** trong đoạn mã nguồn trên thực thi quy trình lan truyền tiến, biến đổi tín hiệu từ lớp đầu vào thành dự đoán ở lớp đầu ra. Về mặt toán học, tiến trình này là một chuỗi các phép biến đổi affine xen kẽ với các hàm kích hoạt phi tuyến.

Tại lớp ẩn, tín hiệu đầu vào tổng hợp tại nơ-ron ẩn thứ j là một phép biến đổi tuyến tính:

$$u_j = a_{0j} + \sum_{i=1}^n a_{ij}x_i$$

Sau đó, tín hiệu này được phóng qua hàm kích hoạt phi tuyến $g(\cdot)$ (như Sigmoid hoặc ReLU) để tạo ra giá trị đầu ra của nơ-ron ẩn:

$$t_j = g(u_j)$$

Việc áp dụng hàm kích hoạt $g(\cdot)$ là bắt buộc. Nếu không có hàm phi tuyến này, toàn bộ mô hình đa lớp sẽ tự suy biến thành một phép biến đổi tuyến tính đơn bậc, triệt tiêu hoàn toàn khả năng học sâu. Hàm Sigmoid (Logistic regression function), được định nghĩa là $g(u) = \frac{1}{1+e^{-u}}$, thường được khảo sát trong toán học vì nó trơn, liên tục, bị chặn, và có đạo hàm dễ dàng tính toán: $g'(u) = g(u)(1 - g(u))$.

Từ lớp ẩn, tín hiệu t_j được lan truyền đến lớp đầu ra. Tổng tín hiệu nhận được tại nơ-ron thứ k của lớp đầu ra là:

$$v_k = b_{0k} + \sum_{j=1}^J b_{jk}t_j$$

Dự đoán cuối cùng của nơ-ron đầu ra là:

$$z_k = g(v_k)$$

Khi kết quả dự đoán z_k được so sánh với nhãn thực tế y_k , ta có thể đánh giá mức độ sai lệch của mạng thông qua hàm mất mát $\mathcal{L}(w)$. Trong phân tích toán học cơ sở, hàm bình phương tối thiểu thường được sử dụng cho một điểm dữ liệu: $E = \frac{1}{2} \sum_{k=1}^m (z_k - y_k)^2$.

Chương 3

GIẢI PHẪU THUẬT TOÁN LAN TRUYỀN NGƯỢC (BACKPROPAGATION)

Để mô hình học hỏi từ dữ liệu, các tham số (ma trận A và B) phải được tinh chỉnh liên tục. Việc này đòi hỏi hệ thống phải tính toán được véc-tơ đạo hàm của hàm lỗi ∇E theo từng trọng số riêng lẻ $\frac{\partial E}{\partial a_{ij}}$ và $\frac{\partial E}{\partial b_{jk}}$. Thuật toán Lan truyền ngược (Error Back-Propagation) được thiết kế dựa trên Quy tắc Chuỗi (Chain Rule) của giải tích vi phân để giải quyết bài toán này một cách tối ưu.

Lan truyền ngược bắt đầu bằng việc đánh giá hàm mất mát ở lớp cuối cùng, sau đó truy ngược sai số trở lại các lớp trước. Cấu trúc tính toán giải tích của tiến trình này được thiết lập như sau :

3.1. Đạo hàm tại Lớp Đầu ra (Cập nhật ma trận B)

Áp dụng quy tắc chuỗi cho trọng số nối từ nơ-ron ẩn j đến nơ-ron đầu ra k :

$$\frac{\partial E}{\partial b_{jk}} = \frac{\partial E}{\partial z_k} \frac{\partial z_k}{\partial v_k} \frac{\partial v_k}{\partial b_{jk}}$$

Từng thành phần được tính toán cụ thể từ định nghĩa của quá trình lan truyền tiến:

1. **Đạo hàm của hàm lỗi theo dự đoán:** Với $E = \frac{1}{2}(z_k - y_k)^2$, ta có $\frac{\partial E}{\partial z_k} = z_k - y_k$.
2. **Đạo hàm của hàm kích hoạt:** Do $z_k = g(v_k)$ (với giả thiết hàm Sigmoid), ta có $\frac{\partial z_k}{\partial v_k} = g'(v_k) = z_k(1 - z_k)$.
3. **Đạo hàm của phép biến đổi affine:** Do $v_k = b_{0k} + \sum_{j=1}^J b_{jk}t_j$, ta có $\frac{\partial v_k}{\partial b_{jk}} = t_j$ (với $j > 0$) và $\frac{\partial v_k}{\partial b_{0k}} = 1$.

Bằng cách gom cụm hai thành phần đầu tiên, ta định nghĩa một đại lượng lỗi cục bộ tại nơ-ron đầu ra k , ký hiệu là p_k :

$$p_k = \frac{\partial E}{\partial z_k} \frac{\partial z_k}{\partial v_k} = (z_k - y_k)z_k(1 - z_k)$$

Như vậy, công thức đạo hàm riêng cho trọng số lớp đầu ra được đơn giản hóa thành:

$$\frac{\partial E}{\partial b_{jk}} = \begin{cases} p_k, & j = 0 \\ p_k \cdot t_j, & j > 0 \end{cases}$$

Nhận xét toán học: Đạo hàm của một trọng số nối từ nơ-ron j đến nơ-ron k chính là tích của giá trị tín hiệu phát ra từ nơ-ron j (t_j) và giá trị lỗi cục bộ đo được tại nơ-ron k (p_k).

3.2. Đạo hàm tại Lớp Ẩn (Cập nhật ma trận A)

Việc tính toán đạo hàm cho trọng số $a_{\{ij\}}$ kết nối từ nơ-ron đầu vào i đến nơ-ron ẩn j phức tạp hơn, vì lỗi từ nơ-ron ẩn j ảnh hưởng đến toàn bộ các nơ-ron đầu ra $k \in \{1, \dots, m\}$. Theo quy tắc chuỗi tổng quát:

$$\frac{\partial E}{\partial a_{ij}} = \frac{\partial E}{\partial t_j} \frac{\partial t_j}{\partial u_j} \frac{\partial u_j}{\partial a_{ij}}$$

Trong đó, đạo hàm của hàm lỗi theo tín hiệu nơ-ron ẩn t_j phải tổng hợp sai số từ tất cả các đầu ra:

$$\frac{\partial E}{\partial t_j} = \sum_{k=1}^m \frac{\partial E}{\partial z_k} \frac{\partial z_k}{\partial v_k} \frac{\partial v_k}{\partial t_j} = \sum_{k=1}^m p_k \frac{\partial v_k}{\partial t_j}$$

Vì $v_k = b_{0k} + \sum b_{jk} t_j$, ta có $\frac{\partial v_k}{\partial t_j} = b_{jk}$. Do đó:

$$\frac{\partial E}{\partial t_j} = \sum_{k=1}^m p_k b_{jk}$$

Tiếp tục chuỗi vi phân qua hàm kích hoạt của lớp ẩn: $\frac{\partial t_j}{\partial u_j} = t_j(1 - t_j)$.

Tương tự như lớp đầu ra, ta định nghĩa đại lượng lỗi cục bộ tại nơ-ron ẩn j , ký hiệu là q_j :

$$q_j = \frac{\partial E}{\partial t_j} \frac{\partial t_j}{\partial u_j} = \left(\sum_{k=1}^m p_k b_{jk} \right) t_j(1 - t_j)$$

Cuối cùng, kết hợp với $\frac{\partial u_j}{\partial a_{ij}} = x_i$, đạo hàm riêng phần cho trọng số lớp ẩn là:

$$\frac{\partial E}{\partial a_{ij}} = \begin{cases} q_j, i = 0 \\ q_j \cdot x_j, i > 0 \end{cases}$$

Tiến trình lan truyền ngược yêu cầu các giá trị kích hoạt trung gian (t_j, z_k) được tính toán trong lúc lan truyền tiến phải được lưu trữ tạm thời trong bộ nhớ đệm (caching). Điều này giúp tiết kiệm tối đa khối lượng tính toán, nhưng đồng thời cũng tiêu tốn dung lượng bộ nhớ tuyến tính theo độ sâu của mạng.

3.3. Cơ chế Hoạt động trong Trình Tối ưu hóa (Optimizer) của PyTorch

Mặc dù các công thức đạo hàm trên mang tính chất nền tảng tuyệt đối, người lập trình mô hình học sâu hiện đại không cần thiết lập thủ công từng phương trình vi phân đó. Thư viện PyTorch cung cấp một công cụ tính toán tự động gọi là công cụ autograd (autograd engine). Công cụ này tự động dựng nên một đồ thị tính toán định hướng có chu trình (computational graph) song song với quá trình lan truyền tiến.

Bảng dưới đây tóm tắt sự tương quan giữa các lệnh cốt lõi trong vòng lặp huấn luyện PyTorch và các cơ chế toán học vi phân tương ứng:

Lệnh trong PyTorch	Ý nghĩa Toán học và Xử lý Đồ thị Tính toán
<code>loss = criterion(output, target)</code>	Đánh giá hàm mục tiêu $\mathcal{L}(w) = \frac{1}{N} \sum E^{(i)}$. Tạo nút gốc cho quá trình truy xuất ngược trên đồ thị tính toán. Lỗi ở bước này thường khởi điểm rất cao.
<code>loss.backward()</code>	Kích hoạt công cụ autograd. Tự động áp dụng Quy tắc Chuỗi (như đã chứng minh với p_k và q_j) để tính toán gradient cho tất cả các trọng số có thuộc tính <code>requires_grad=True</code> . Các đạo hàm $\frac{\partial E}{\partial a_{ij}}$ và $\frac{\partial E}{\partial b_{jk}}$ được lưu vào thuộc tính <code>.grad</code> của mỗi biến tensor.

<code>optimizer.step()</code>	Gọi thuật toán tối ưu. Trong cấu hình chuẩn của SGD (Stochastic Gradient Descent), hàm này thực thi công thức: $w^{(t+1)} = w^{(t)} - \alpha \cdot w.grad$. Các tham số hiện hành được tinh chỉnh theo hướng giảm của véc-tơ đạo hàm.
<code>optimizer.zero_grad()</code>	Xóa sạch bộ đệm đạo hàm <code>.grad</code> từ chu kỳ trước. Nếu không xóa, PyTorch sẽ cộng dồn (accumulate) gradient của chu kỳ mới vào chu kỳ cũ, làm hỏng hướng tiệm cận của mô hình.

Quá trình `loss.backward()` tương đương với giai đoạn tính toán và thu thập các ma trận lỗi, trong khi `optimizer.step()` chính là quá trình cập nhật thích nghi trọng số theo lược đồ Heuristic Gradient Descent. Mô hình lặp lại quá trình này trên nhiều chu kỳ (Epochs) cho đến khi sai số giảm đến mức tiệm cận tối ưu. Tuy nhiên, việc sử dụng GD với hệ số học α cố định mang theo nhiều hạn chế về hiệu năng: thuật toán bị mù về mặt hình học toàn cục, gặp hiện tượng dao động (oscillation) mạnh khi đi vào các thung lũng hẹp, và có xu hướng lết chậm chạp ở những vùng bình nguyên thoải thoải (plateaus) của hàm mất mát.

Chương 4

TỐI ƯU HÓA NÂNG CAO: CHUYỂN DỊCH TỪ GRADIENT DESCENT SANG PHƯƠNG PHÁP PARABOL

Để vượt qua những giới hạn của phương pháp tối ưu chỉ dựa trên thông tin đạo hàm bậc nhất (first-order methods) như Gradient Descent, toán học số cung cấp các công cụ khai thác đạo hàm bậc hai (thông tin độ cong) của hàm mục tiêu. Tuy nhiên, việc tính toán trực tiếp ma trận Hesse khổng lồ $\nabla^2 \mathcal{L}(w)$ đối với mạng nơ-ron hàng triệu tham số là bất khả thi về mặt tài nguyên. Giải pháp hoàn hảo cho bài toán này là sử dụng thuật toán xấp xỉ bậc hai dựa trên Phương pháp Nội suy Parabol Liên tiếp (Successive Parabolic Interpolation), hay còn gọi là Phương pháp Brent, mà hiện thân nổi tiếng nhất trong học sâu chính là thuật toán **QuickProp**.

4.1. Cơ sở Toán học của Phương pháp Parabol (Brent's Method)

Ý tưởng cốt lõi của nội suy Parabol là xấp xỉ bề mặt của hàm mục tiêu $f(x)$ tại lân cận của điểm đang xét bằng một hàm toàn phương (parabol). Do tính chất lồi tuyệt đối của hàm toàn phương, ta có thể phóng thẳng bước nhảy tới ngay điểm cực tiểu của parabol xấp xỉ này, thay vì lết chậm chạp bằng hằng số α .

Trong không gian hàm 1 chiều, một parabol duy nhất được xác định qua 3 điểm phân biệt $(a, f(a))$, $(b, f(b))$, và $(c, f(c))$. Tại điểm cực trị của parabol, đạo hàm của nó sẽ bằng không. Bằng cách giải phương trình đạo hàm $f'(x) = 0$ cho đa thức bậc hai đi qua 3 điểm trên, toán học tối ưu đã cung cấp một công thức giải tích đóng cho vị trí cực tiểu x :

$$x = b - \frac{1}{2} \frac{(b-a)^2[f(b)-f(c)] - (b-c)^2[f(b)-f(a)]}{(b-a)[f(b)-f(c)] - (b-c)[f(b)-f(a)]}$$

Phương pháp Parabol tạo ra một chuỗi tìm kiếm đệ quy: thu thập 3 điểm, tính ra điểm x ở đáy parabol, đánh giá hàm tại x , sau đó loại bỏ điểm cũ nhất và lặp lại việc dựng parabol mới với 3 điểm cập nhật. Khi áp dụng cho không gian nhiều chiều $\mathbb{R}^m$, thuật toán có thể được sử dụng tuần tự dọc theo hướng của véc-tơ đạo hàm hoặc một hướng tìm kiếm chỉ định.

4.2. Ứng dụng Parabol vào Mạng Nơ-ron: Thuật toán QuickProp

Tuy nhiên, việc lượng giá hàm mất mát $\mathcal{L}(w)$ qua 3 bộ trọng số phân biệt ở mỗi chu kỳ học là quá tốn kém. Dựa trên ý tưởng Parabol, Scott Fahlman (1988) đã đưa ra thuật toán Quick Propagation (QuickProp) - thay vì nội suy dựa trên 3 điểm của hàm giá trị, QuickProp thiết lập parabol xấp xỉ bằng cách sử dụng 2 điểm của **véc-tơ đạo hàm**.

Nếu $\mathcal{L}(w)$ được xấp xỉ là một parabol, thì đạo hàm $\Delta = \frac{\partial \mathcal{L}}{\partial w}$ của nó là một hàm tuyến tính (đường thẳng). Mục tiêu của thuật toán là tìm điểm trọng số mà tại đó $\Delta = 0$.

Giả sử ta xét một trọng số cụ thể w_i . Định nghĩa các ký hiệu qua hai bước lặp $t-1$ và t :

- $\Delta^{(t-1)}$ và $\Delta^{(t)}$: Giá trị đạo hàm tại bước trước và bước hiện tại.
- $C^{(t-1)} = w^{(t)} - w^{(t-1)}$: Biến thiên trọng số (bước nhảy) đã thực hiện ở bước trước.
- $C^{(t)}$: Biến thiên trọng thích nghi cần tính cho bước lặp tiếp theo.

Biểu diễn quan hệ tuyến tính của đạo hàm trên một đồ thị với trục hoành là không gian tham số w và trục tung là giá trị đạo hàm Δ . Hai điểm đã biết là $(\Delta^{(t-1)}, w^{(t-1)})$ và $(\Delta^{(t)}, w^{(t)})$.

Độ dốc của đường thẳng nối hai điểm này (tương đương với đạo hàm bậc hai xấp xỉ rời rạc) được tính bằng tỷ số giữa sự thay đổi đạo hàm và sự thay đổi trọng số.

Dựa vào tam giác đồng dạng, tỷ lệ suy giảm của đạo hàm được thể hiện qua phương trình tỷ lệ biến thiên trọng số:

$$\frac{C^{(t)}}{C^{(t-1)}} = \frac{\Delta^{(t)} - \Delta^{(t+1)}}{\Delta^{(t-1)} - \Delta^{(t)}}$$

Vì mục tiêu là hướng đến cực tiểu của parabol, nơi mà đạo hàm phải triệt tiêu, ta đặt $\Delta^{(t+1)} = 0$. Khi đó, biến thiên trọng thích nghi cho chu kỳ tiếp theo được xác định bằng công thức cốt lõi của QuickProp:

$$C^{(t)} = \frac{\Delta^{(t)}}{\Delta^{(t-1)} - \Delta^{(t)}} C^{(t-1)}$$

Sự chuyển dịch này là đặc biệt xuất sắc. Phương pháp cập nhật trọng số tiêu chuẩn $w^{(t+1)} = w^{(t)} - \alpha \Delta^{(t)}$ đã hoàn toàn bị thay thế bởi công thức:

$$w^{(t+1)} = w^{(t)} + C^{(t)}$$

Thay vì bị trói buộc bởi hệ số học tĩnh α , mạng nơ-ron giờ đây sở hữu một bước nhảy $C^{(t)}$ thích ứng linh hoạt cho từng trọng số độc lập. Thuật toán QuickProp lợi dụng quán tính quá khứ ($C^{(t-1)}$) và sử dụng hệ số gia tốc $\frac{\Delta^{(t)}}{\Delta^{(t-1)} - \Delta^{(t)}}$ để điều hướng. Nếu đạo hàm duy trì cùng hướng nhưng giảm độ lớn, thuật toán sẽ tự tin phóng to bước nhảy. Nếu đạo hàm đổi dấu (dấu hiệu đã băng qua rãnh cực tiểu), hệ số này tự động mang giá trị âm nhỏ để phanh hệ thống lại.

4.3. So sánh Tương quan Đặc tính Toán học

Bảng dưới đây tóm tắt sự khác biệt về mặt bản chất hình học giữa Gradient Descent và Phương pháp Parabol (QuickProp) trong việc cập nhật tham số mạng nơ-ron:

Tiêu chí	Gradient Descent (Đạo hàm Bậc Một)	Phương pháp Parabol / QuickProp (Xấp xỉ Bậc Hai)
Cơ sở Toán học	Trượt theo độ dốc tiếp tuyến (leo đồi đảo ngược).	Nội suy và hội tụ thẳng đến đáy của bề mặt parabol xấp xỉ.
Bước cập nhật	$C^{(t)} = -\alpha \nabla \mathcal{L}(w^{(t)})$	$C^{(t)} = \frac{\Delta^{(t)}}{\Delta^{(t-1)} - \Delta^{(t)}} C^{(t-1)}$
Tính Thích nghi	Phụ thuộc hoàn toàn vào hệ số tĩnh α . Khó thiết lập tối ưu.	Tự động sinh ra hệ số học thích nghi cho mỗi trọng số riêng lẻ dựa vào lịch sử đạo hàm.
Sự kiện Biện dị	Mắc kẹt tại vùng bình nguyên phẳng, dao động dữ dội tại khe hẹp.	Nếu $\Delta^{(t-1)} \approx \Delta^{(t)}$, mẫu số tiến về 0, gây bùng nổ bước nhảy (Exploding Step). Yêu cầu cơ chế kìm hãm fallback.

Mặc dù có ưu điểm đột phá về tốc độ hội tụ, công thức QuickProp tồn tại một điểm kì dị toán học (singularity point): khi giá trị đạo hàm của chu kỳ hiện tại tương đương với chu kỳ trước

$(\Delta^{(t-1)} = \Delta^{(t)})$, mẫu số sẽ bằng 0 khiến bước cập nhật trở nên vô hạn. Tại những thời điểm này, thuật toán phải được trang bị cơ chế tự động thoái lui (fallback) về phương pháp Gradient Descent tiêu chuẩn $C^{(t)} = -\alpha \Delta^{(t)}$ để duy trì độ ổn định.

Chương 5

TÍCH HỢP VÀ THAY THẾ LOGIC PARABOL VÀO MÃ NGUỒN THỰC TẾ

Từ góc độ lập trình, việc ứng dụng lý thuyết toán học của phương pháp Parabol (QuickProp) vào mô hình MLP được thực hiện bằng cách đóng băng phương thức `optimizer.step()` của hệ sinh thái PyTorch. Do PyTorch bảo vệ các tensor có yêu cầu gradient, mọi thao tác ghi đè tự do lên trọng số mạng phải được bao bọc bên trong ngữ cảnh vi phân tắt `torch.no_grad()`.

Quá trình thay thế Gradient Descent bằng Phương pháp Parabol được diễn giải toán học thông qua kịch bản lập trình sau:

5.1. Khởi tạo Bức tranh Quá khứ

Trái ngược với GD, phương pháp Parabol yêu cầu duy trì hai đại lượng lịch sử: Đạo hàm chu kỳ trước ($\Delta^{(t-1)}$) và Biến thiên trọng số chu kỳ trước ($C^{(t-1)}$).

```
# Khởi tạo danh sách tensor lưu trữ lịch sử cho toàn bộ kiến trúc mạng
prev_grads = [torch.zeros_like(p) for p in model.parameters()]
prev_steps = [torch.zeros_like(p) for p in model.parameters()]
alpha = 0.01 # Hệ số học cơ sở dùng cho bước fallback
```

5.2. Lan truyền Tiến và Kích hoạt Autograd

Mô hình giữ nguyên cơ chế hoạt động cho việc thu thập đạo hàm $\Delta^{(t)}$. Công cụ tự động của PyTorch xây dựng đồ thị đạo hàm riêng dựa vào Quy tắc Chuỗi của phép vi phân.

```
output = model(data)
loss = criterion(output, target)
# Kích hoạt tính toán gradient p_k và q_j, gán vào p.grad
loss.backward()
```

5.3. Triển khai Logic Toán học Cập nhật Parabol

Kịch bản sau hoàn toàn thay thế hành vi của `optimizer.step()`, chuyển dịch mô hình tính toán từ bậc một sang ngoại suy bậc hai.

```
with torch.no_grad(): # Vô hiệu hóa autograd để tinh chỉnh tham số thủ công
    for i, p in enumerate(model.parameters()):
        if p.grad is not None:
            # Thu thập biến trạng thái từ bộ đệm
            delta_t = p.grad # \Delta^{(t)}
```

```

delta_t_minus_1 = prev_grads[i]    #  $\Delta^{(t-1)}$ 
c_t_minus_1 = prev_steps[i]        #  $C^{(t-1)}$ 

# Tính toán độ dốc đạo hàm (Mẫu số của QuickProp)
# Sự chênh lệch này biểu thị độ cong của mặt Parabol
grad_diff = delta_t_minus_1 - delta_t

# Khởi tạo Tensor chứa bước cập nhật  $C^{(t)}$ 
c_t = torch.zeros_like(p)

# --- CƠ CHẾ XỬ LÝ BIÊN (Fallback) ---
# Giải quyết bài toán chia cho 0 khi  $\Delta^{(t-1)} = \Delta^{(t)}$ 
# Hoặc khi thuật toán mới chạy ở bước t=0 ( $c_{t\_minus\_1} == 0$ )
mask_zero = (torch.abs(grad_diff) < 1e-6) | (c_t_minus_1 == 0)
# Tại những điểm kì dị này, thuật toán rút lui về Gradient Descent

chuẩn
c_t[mask_zero] = -alpha * delta_t[mask_zero]

# --- CÔNG THỨC PARABOL CỐT LỖI (QuickProp) ---
#  $C^{(t)} = * C^{(t-1)}$ 
mask_valid = ~mask_zero
c_t[mask_valid] = (delta_t[mask_valid] / grad_diff[mask_valid])
* c_t_minus_1[mask_valid]

# Cập nhật trọng số của mạng
#  $w^{(t+1)} = w^{(t)} + C^{(t)}$ 
p.add_(c_t)

# Ghi đè bộ nhớ đệm lịch sử cho chu kỳ t+1
prev_grads[i].copy_(delta_t)
prev_steps[i].copy_(c_t)

# Xóa đạo hàm cục bộ để chống cộng dồn ở epoch sau
model.zero_grad()

```

Kịch bản triển khai này phản ánh sự tương giao hoàn hảo giữa cấu trúc dữ liệu và thuật toán. Mỗi tensor trọng số p không còn lặp lại quỹ đạo đạo hàm một cách mù quáng, mà trở thành một thực thể có trí nhớ. Bằng cách lưu trữ và đối chiếu các tín hiệu biến thiên không gian, phép toán vi phân bậc một được nội suy để khai thác được cấu trúc xấp xỉ bậc hai của hàm mất mát.

KẾT LUẬN

Khoa học máy tính, ở tầng nhận thức cơ bản, là sự kết hợp giữa tổ chức dữ liệu và thiết lập chỉ thị toán học. Các mạng nơ-ron đa lớp, qua việc kết cấu nên một không gian mô hình phi tuyến tính vì dựa vào vô số phép toán affine liên hợp với các hàm kích hoạt Sigmoid hay ReLU, đã mở ra khả năng xấp xỉ bất kỳ miền dữ liệu phức tạp nào. Trung tâm của năng lực này nằm ở thuật toán Lan truyền Ngược (Backpropagation). Bằng việc thao túng Quy tắc Chuỗi của giải tích, thuật toán phân rã được ma trận gradient $\nabla \mathcal{L}$ khổng lồ thành vô số thành tố p_k và q_j tại cấp độ từng nơ-ron.

Tuy nhiên, việc sở hữu bản đồ đạo hàm không tự động đảm bảo một lộ trình hội tụ tối ưu. Phương pháp tìm kiếm Heuristic cơ bản như Gradient Descent thể hiện giới hạn nghiêm trọng trong việc điều hướng trên một không gian tham số phi tuyến đa chiều. Giải pháp thay thế cơ chế tối ưu bằng Phương pháp Parabol (thông qua dẫn xuất QuickProp) chứng minh sự vượt trội của tiếp cận toán học xấp xỉ bậc hai.

Thông qua việc đánh giá tỷ lệ suy giảm của véc-tơ đạo hàm qua hai bước lặp tuần tự, thuật toán Parabol đã phá vỡ sự kìm hãm của hệ số học hằng số α , cấp cho mỗi tham số mô hình một khả năng tự thích nghi chiều dài bước nhảy. Sự thay thế mã nguồn `optimizer.step()` bằng một cấu trúc tensor nội suy không chỉ thỏa mãn phương trình cực tiểu của mặt parabol hình học mà còn khẳng định nguyên lý: sức mạnh của Trí tuệ Nhân tạo không chỉ nằm ở khối lượng dữ liệu nó hấp thụ, mà còn nằm ở sự tinh tế của các kiến trúc toán học mà nó sử dụng để tối ưu hóa chính không gian nhận thức của mình.

DANH MỤC TÀI LIỆU THAM KHẢO

- Badjideh, A. (2021). *Multi-Layer-Perceptron-Pytorch [Source code]*. Retrieved from GitHub: <https://github.com/GLAZERadr/Multi-Layer-Perceptron-Pytorch>
- ChrisCC. (2018). *MNIST-MLP-PyTorch [Source code]*. Retrieved from Kaggle: <https://www.kaggle.com/code/chriscc/mnist-mlp-pytorch>
- Christian Versloot. (2021). *Creating a Multilayer Perceptron with PyTorch and Lightning [Technical Article]*. Retrieved from MachineCurve: <https://github.com/christianversloot/machine-learning-articles/blob/main/creating-a-multilayer-perceptron-with-pytorch-and-lightning.md>
- CSC - IT Center for Science. (2023). *pytorch-mnist-mlp.ipynb [Source code]*. Retrieved from GitHub: <https://github.com/CSCfi/machine-learning-scripts/blob/master/notebooks/pytorch-mnist-mlp.ipynb>
- GeeksforGeeks. (2023). *PyTorch – Connection between loss.backward() and optimizer.step() [Technical Article]*. Retrieved from GeeksforGeeks: <https://www.geeksforgeeks.org/deep-learning/pytorch-connection-between-lossbackward-and-optimizerstep/>
- Intellipaat. (2024). *Backpropagation in Neural Networks | PyTorch Backpropagation - Theory and Example [Video]*. Retrieved from YouTube: <https://www.youtube.com/watch?v=guFaqnqrnzS>
- Modroczký, V. (2023). *MLP-Fashion-MNIST [Source code]*. Retrieved from GitHub: <https://github.com/vktr274/MLP-Fashion-MNIST>
- Nguyen, T. D. (2025). *AI_Sec_Chapter1_OptnAI*.
- Nguyen, T. D. (2025). *AI_Sec_Chapter2_AlnML*.
- Nguyen, T. D. (2025). *Maths4AI Preface*.
- Noel Benjamin. (2023). *Breaking down Backpropagation in PyTorch [Technical Article]*. Retrieved from Medium: <https://medium.com/@noel.benji/breaking-down-backpropagation-in-pytorch-3762ea107d3a>

PyTorch Community. (2019). *Manual Back-propagation in Neural Network [Forum Discussion]*. Retrieved from PyTorch Forums: <https://discuss.pytorch.org/t/manual-back-propagation-in-neural-network/49368>

PyTorch Contributors. (2024). *Learning PyTorch with Examples [Technical Tutorial]*. Retrieved from PyTorch Tutorials: https://docs.pytorch.org/tutorials/beginner/pytorch_with_examples.html

PyTorch Contributors. (2024). *torch.optim — PyTorch Documentation*. Retrieved from PyTorch Stable Documentation: <https://docs.pytorch.org/docs/stable/optim.html>